

In this exercise, you're going to practice adding CSS to an HTML file using all three methods: external CSS, internal CSS, and inline CSS. You should only be using type selectors for this exercise when adding styles via the external and internal methods. You should also use keywords for colors (e.g. "blue") instead of using RGB or HEX values.

There are three elements for you to add styles to, each of which uses a different method of adding CSS to it, as noted in the outcome image below. All other exercises in this section will have a CSS file provided and linked for you, but for this exercise you will have to create the file and link it in the HTML file yourself. This is all about practicing using these different methods and getting the syntax right.

The properties you need to add to each element are:

- * ``div``: a red background, white text, a font size of 32px, center aligned, and bold
- * ``p``: a green background, white text, and a font size of 18px
- * ``button``: an orange background and a font size of 18px

Style me via the external method!

I would like to be styled with the internal method, please.

Inline Method

Class and ID Selectors

Knowing how to add class and ID attributes to HTML elements, as well as use their respective selectors, is invaluable. It's important to practice using them.

There are several elements in the HTML file provided, which you will have to add either class or ID attributes to, as noted in the outcome image below. You will then have to add rules in the CSS file provided using the correct selector syntax. Look over the outcome image carefully, and try to keep in mind which elements look similarly styled (classes), which ones may be completely unique from the rest (ID), and which ones have slight variations from others (multiple classes).

It isn't entirely important which class or ID values you use, as the focus here is on being able to add the attributes and use the correct selector syntax to style elements. For the colors in this exercise, try using a non-keyword value (RGB, HEX, or HSL). The properties you need to add to each element are:

- *All odd numbered elements: a light red/pink background, and a list of fonts containing ``Verdana`` and ``DejaVu Sans`` with ``sans-serif`` as a fallback
- *The second element: blue text and a font size of 36px
- *The third element: in addition to the styles for all odd numbered elements, add a font size of 24px
- *The fourth element: a light green background, a font size of 24px, and bold

Number 1 - I'm a class!

Number 2 - I'm one ID.

Number 3 - I'm a class, but cooler!

Number 4 - I'm another ID.

Number 5 - I'm a class!

Grouping Selectors

Let's build a little off the previous exercise. Here, you're going to give two elements each a unique class name, then add rules for styles that both elements share as well as their own unique styles. Make sure you take a good look at the outcome image below to see exactly what is unique about each element, and what both elements have in common.

This will help you further practice adding classes and using class selectors, so be sure you add the class attribute in the HTML file. For the remainder of these exercises, the format of any colors is entirely up to you; we trust you'll practice using the different values! The properties you need to add to each element are:

*The first element: a black background and white text

*The second element: a yellow background

Both elements: a font size of 28px and a list of fonts containing `Helvetica` and `Times New Roman`, with `sans-serif` as a fallback



Chaining Selectors

Credits for the images in this exercise go to [Katho Mutodo](https://linktr.ee/photobykatho_) and [Andrea Piacquadio](https://www.pexels.com/@olly?utm_content=attributionCopyText&utm_medium=referral&utm_source=pexels).

With this exercise, we've provided you with a complete HTML file and a CSS file to work with. The purpose of this exercise is to focus on understanding how to chain different selectors, rather than solely adding attributes.

We have two images for you to style, each with two class names, where one of the class names is shared. The goal here is to chain the selectors for both elements, so that each have a unique style applied, despite using a shared class selector. For example, you want an element that has both X and Y to have one set of styles, while an element with X and Z has a completely different set of styles. We included the original images as well, so that you can see how the styles you will be adding look in comparison, so do not add any styles to them.

The properties you need to add to each element are:

* Make the element with both the `avatar` and `proportioned` classes 300 pixels wide. We want it to automatically retain its original square proportions, so don't hardcode in a pixel value for its height.

* Make the element with both the `avatar` and `distorted` classes 200 pixels wide, then make its height twice as big as its width (here you should hardcode in a pixel value).



Descendant Combinator

Understanding how combinators work can become a lot easier when you start playing around with them and see what exactly is affected by them versus what isn't.

The goal of this exercise is to apply styles to elements that are descendants of another element, while leaving elements that *aren't* descendants of that element unstyled.

You can use either type or class selectors for this exercise; use whichever you may feel you want to practice with more. The HTML file is set up (so no need to edit anything in it) such that any combination of selectors will work, so if you're feeling adventurous you can even try combining a type and class selector for the descendant combinator.

The properties you need to add are:

* Only the `p`` elements that are descendants of the `div`` element should have a yellow background, red text, a font size of 20px, and center aligned.

This should be styled.

This should be unstyled.

This should be unstyled.

This should be styled.

This should be styled.